LOVELY PROFESSIONAL UNIVERSITY, SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CSE476

# Agentic AI and Intelligent Automation

Lecture Zero. What this course is, why it exists,
and what you will be able to build by the end of it.

Rohit Bharti

SECTION 00

# Let us start with a demo, not a syllabus

Two minutes. Then we talk.

# Watch this before I explain anything

I am going to give a program a goal. Not a question. A goal.

I will not tell it which steps to take. I will not tell it which tools to use. I will give it the outcome I want and then I will stop typing.

**WHAT TO WATCH FOR**

Count how many times it stops and thinks between actions. That number is the whole difference between what you have used before and what you are about to learn.

# The goal I gave it

> *"Find out which of our three suppliers had the worst on time delivery last quarter, then draft the escalation email."*

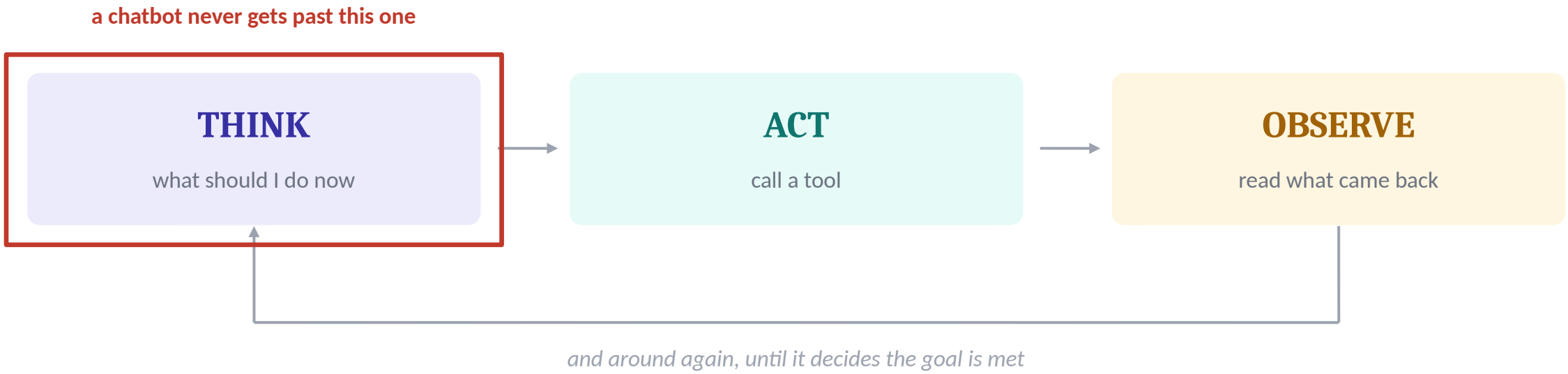
Notice what is missing from that sentence. I did not say where the data lives. I did not say to open the spreadsheet first. I did not say to compare before writing. I did not say what an escalation email looks like.

A search box would return links. A chatbot would ask me to paste the data. This thing went and got it.

# What it actually did, step by step

| | |
|---|---|
| THINK | I need delivery records. I do not have them. There is a database tool. |
| ACT | Query the supplier delivery table for last quarter. |
| OBSERVE | Three suppliers returned. Percentages differ. |
| THINK | Rao Logistics is lowest at 71 percent. That is the answer to part one. |
| ACT | Draft an escalation email addressed to Rao Logistics. |
| OBSERVE | Draft produced. Both parts of the goal are now done. Stop. |

# Three words. That is the engine.




A chatbot runs this once and stops. An agent runs it again, and again, deciding each time whether it is finished.

**HOLD ON TO THIS**

The red box is on THINK, not on ACT. Acting is the part that looks impressive. Reasoning between actions is the part that actually makes it an agent.

# So what is the actual difference?

WHAT YOU HAVE USED

## A chatbot answers

One question in, one answer out. It knows things. It cannot do things. If it needs your data, it asks you to paste it.

WHAT WE ARE BUILDING

## An agent finishes

One goal in, a completed task out. It has tools, it chooses which to use, it reads the result, and it decides on its own when the work is done.

HOLD ON TO THIS

The difference is not intelligence. Both use the same underlying model. The difference is permission to act and the loop that lets it keep acting.

# That demo is this entire course

Everything for the next fifty hours sits somewhere inside that loop.

How the model decides what to do next. How you give it tools safely. How you stop it looping forever. How you know it is telling the truth. How you get it out of your laptop and into something a company can actually run.

**ONE HONEST WARNING**

That demo took me about forty lines of code. Getting the same thing to work reliably, for a thousand users, without leaking data or inventing facts, takes considerably more. That gap is where most of this course lives, and it is also where the jobs are.

SECTION 01

# What actually changed

And why this course did not exist three years ago.

# The last four years, compressed

| | | |
|---|---|---|
| 2022 | **It could talk** | Fluent text. Impressive. Completely sealed off from the world. |
| 2023 | **It could be pointed at your data** | Retrieval. Now it could answer from documents it had never memorised. |
| 2024 | **It could call your functions** | Tool calling arrives. The model can now trigger real code. |
| 2025 | **It could run a loop** | Frameworks make think, act, observe routine instead of research. |
| **2026** | **It has to behave in production** | The hard questions arrive: cost, safety, identity, audit, governance. |

**you are here**

HOW I WILL TEACH EVERY TOPIC

# The layman version first

## Think of hiring someone for your shop.

THE ENCYCLOPEDIA

### You ask, it tells

A very well read person sitting on a chair. Ask anything, get a good answer. But it will not get up. It will not go to the godown and count the stock for you.

THE EMPLOYEE

### You ask, it goes

Same knowledge. But this one has the godown keys, the ledger, and the phone. Tell it what you need and it walks off and comes back with the answer.

NOW THE TECHNICAL PART

The keys, the ledger and the phone are called tools. The walking off and coming back is called the agent loop. Everything else in this course is detail on top of those two ideas.

# Why suddenly everyone at once

- Models got reliable enough at following a schema. Tool calling only works if the model returns exactly the structure your code expects, every time. That got solved.
- Context windows got long enough to hold a working memory of what the agent has already tried.
- Inference got cheap enough that running the loop twenty times instead of once stopped being unaffordable.
- Frameworks arrived. What was six months of research code in 2023 is now an import statement.
- And crucially, the money arrived. Enterprises stopped experimenting and started budgeting.

**HOLD ON TO THIS**

None of these five is an intelligence breakthrough. They are engineering breakthroughs. That is a hopeful thing for you, because engineering is learnable.

# The part the hype does not mention

**Most agent demos on the internet work once, on the demo machine, with the demo question.**

FAILURE 1

### It loops forever

No termination condition. It burns forty API calls deciding it is not done yet. You find out from the bill.

FAILURE 2

### It confidently lies

The retrieval pulled the wrong paragraph. The agent does not know that. It answers with total confidence.

FAILURE 3

### It gets talked into things

A document it reads contains an instruction. It obeys the document instead of you. This has a name: prompt injection.

WHICH IS WHY

Units 5 and 6, fourteen of our fifty hours, are about exactly these three failures. Not because the syllabus says so, but because this is what separates a student project from something a company will pay for.

SECTION 02

# What this course is, and what it is not

So nobody is surprised in November.

# The promise, in one sentence

**By December you will have built, secured, instrumented and deployed a working multi agent system, and you will be able to explain every design decision in it to an interviewer.**

Read that again and notice the second half. Building it is half the marks. Being able to defend it is the other half, in this room and in every interview after it.

# What this course is

HANDS ON

## Code, not slideware

Twenty one of the fifty hours are you writing and running code. Ten formal practicals plus live builds in almost every session.

CURRENT

## Built this month

Every framework version is verified live before it reaches a slide. This field renames things twice a year and I will tell you when it does.

ENTERPRISE

## Production shaped

Microsoft Foundry, Semantic Kernel, AutoGen, real deployment, real observability, real security. Not toy notebooks.

HOLD ON TO THIS

The whole course is benchmarked against the Microsoft AI Agents professional certificate. If you follow it properly, that credential becomes a short step rather than a fresh start.

# What this course is not

NOT THIS

## Not a prompt writing course

Prompting is one session, not a semester. If you came for prompt tricks you will be disappointed by week two.

NOT THIS

## Not a model training course

We do not train or fine tune models here. We orchestrate them. Different skill, different course.

NOT THIS

## Not a copy the notebook course

I will show you code that is broken, on purpose, before I show you code that works. If you copy without watching, you will not survive the viva.

ALSO WORTH SAYING

This is not a course where attendance alone produces a grade. The practicals compound. Miss the middle of Unit 3 and Unit 4 will not make sense, because Unit 4 builds directly on it.

# What I need from you, and what you get from me

FROM YOU

## Bring the laptop, open the editor

Every session has code. A session where you only watch is a session you will not remember. Also: tell me when something breaks. I would rather fix it in class than read about it in a viva.

FROM ME

## Nothing untested reaches you

Every code cell I hand you has been run against the installed package version, not written from documentation memory. When something is deprecated I will say so and tell you the replacement.

HOLD ON TO THIS

I am learning parts of this stack alongside you. Semantic Kernel and the enterprise governance material are new ground for me too. You will see me check things live. That is the job, not a weakness in it.

SECTION 03

# The map

Six units, fifty hours, one system at the end.

# The whole course on one slide

| | | | |
|---|---|---|---|
| U1 | **Foundations** | What an agent is. Microsoft Foundry. | 8 hrs |
| U2 | **Workflows** | Tool calling, orchestration, automation. | 7 hrs |
| U3 | **Frameworks** | Python, Semantic Kernel, AutoGen, memory, RAG. | 10 hrs |
| U4 | **Multi agent** | Collaboration, delegation, MCP and A2A. | 9 hrs |
| U5 | **Ship it** | Testing, observability, deployment, CI/CD. | 7 hrs |
| U6 | **Trust it** | Responsible AI, security, governance. | 7 hrs |

Fifty hours total. Twenty one of them are hands on code.

EIGHT HOURS

# Unit 1, Foundations of AI Agents and Microsoft Foundry

- What an agent actually is, and the honest boundary between a chatbot and an agent.
- Agent architectures: reactive, goal based, utility based. Perception and action models.
- Planning and reasoning. The ReAct pattern, which you will use for the rest of your career.
- Microsoft Foundry, first contact. Projects, model catalogue, playground, deployments.
- Prompt engineering, specifically for agents rather than for chat.
- A map of the Microsoft AI ecosystem, including which parts are already deprecated.

YOU WILL BUILD

Practical 1: your full development environment across all four access lanes. Practical 2: a conversational hotel information agent, your first working agent.

SEVEN HOURS

# Unit 2, Building Intelligent Agent Workflows

- Workflow versus agency. When a plain deterministic pipeline is the better engineering choice, and how to tell.
- Tool calling mechanics in depth. Schemas, how the model chooses, the execution loop, what happens when a tool fails.
- API integration and connecting an agent to real services.
- Workflow chaining and event driven agent systems.
- Context aware agents that carry state across turns.
- First look at testing and deployment, which Unit 5 then takes seriously.

YOU WILL BUILD

Practical 3: an intelligent workflow automation agent that takes a messy real input and produces a structured, actioned output.

TEN HOURS, THE HEAVIEST UNIT

# Unit 3, Agent Development with Python and Frameworks

- Python for agents: async, typing, Pydantic. I will assume gaps here and fill them rather than pretend.
- Semantic Kernel, the mental model. This is genuinely new thinking, not a renamed version of something you know.
- AutoGen and AG2, conversational agent patterns.
- Memory and state. Short term, long term, session scoped. Why an agent that forgets is nearly useless.
- Prompt templates and structured output you can actually rely on in code.
- Retrieval augmented generation, so the agent can answer from your documents.
- Connecting an agent to a conversational channel using the Microsoft 365 Agents SDK.

YOU WILL BUILD

Practical 4: tool calling and API integration with Semantic Kernel. Practical 5: an agent with working memory and context management.

NINE HOURS

# Unit 4, Multi Agent Systems and Collaboration

- Why use several agents at all, including the honest cases where one agent is simply better.
- Planner and executor architectures. One agent decides, others do.
- Inter agent communication and task delegation.
- Orchestration patterns: sequential, concurrent, group chat, handoff.
- Role based agents, where each agent gets a job description rather than a prompt.
- Open protocols. MCP and A2A. Why the industry needed a standard and what it changed.
- Microsoft Agent Framework 1.0 and where Semantic Kernel and AutoGen are heading.

YOU WILL BUILD

Practical 6: a medical information agent with responsible AI safeguards. Practical 7: a multi agent collaborative workflow system.

SEVEN HOURS, HALF OF IT HANDS ON

# Unit 5, Testing, Monitoring and Deployment

- Why agents are hard to test. The same input gives a different output. So what exactly do you assert?
- Debugging an agent trace, which is a genuinely different skill from debugging a stack trace.
- Observability and telemetry. OpenTelemetry, Azure Monitor, Prometheus and Grafana.
- Hallucination detection and validation. Making the system check its own work.
- Deployment: containers, Azure Container Apps, hosted agents, scale to zero.
- CI/CD and performance optimisation.

YOU WILL BUILD

Practical 8: retrieval augmented generation workflows. Practical 9: monitoring and observability for a running agent.

SEVEN HOURS. THE ONE NOBODY ELSE TEACHES.

# Unit 6, Responsible AI, Security and Governance

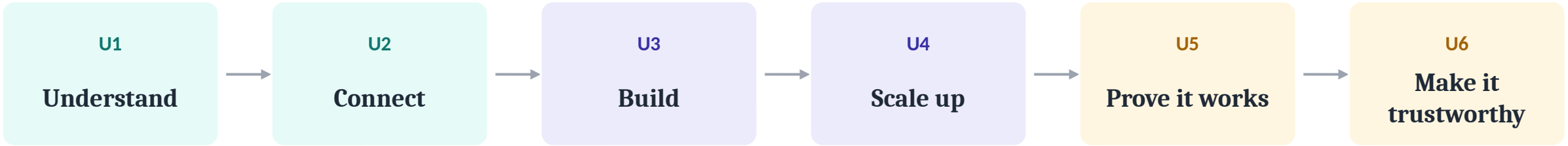
- Responsible AI principles, applied specifically to agents that can take actions.
- Prompt injection, direct and indirect. We will run a successful attack in class before defending against it.
- Authentication and authorisation. Managed identity, least privilege, and the confused deputy problem.
- Content safety, filters and guardrails.
- Compliance, auditing and data residency.
- Enterprise governance models and secure multi agent architecture.

**WHY THIS UNIT IS WORTH YOUR ATTENTION**

Almost every course and tutorial online stops at Unit 4. An agent that works is now a commodity. An agent somebody trusts with real permissions is not. This unit is your differentiator.

# Why this order and not another

**The order is not arbitrary. It follows the shape of a real project.**




A team in industry does exactly this. They understand the problem, connect to the data, build the thing, split it across specialists when it grows, prove it works, and only then are they allowed to give it real permissions on real systems.

You are not learning topics in a syllabus order. You are walking a project lifecycle.

SECTION 04

# What you will actually build

Ten practicals, and one system that survives to the end.

# The ten practicals

| | |
|---|---|
| 01 | Development environment, Foundry and VS Code |
| 02 | Conversational hotel information agent |
| 03 | Intelligent workflow automation agent |
| 04 | Tool calling and API integration, Semantic Kernel |
| 05 | Agent with memory and context management |
| 06 | Medical information agent with responsible AI |
| 07 | Multi agent collaborative workflow system |
| **08** | Retrieval augmented generation workflows |
| **09** | Monitoring and observability for agents |
| **10** | Secure deployment and governance |

**The last three are highlighted for a reason. Next slide.**

# Practicals 8, 9 and 10 are one system, not three

Most courses hand you ten disconnected notebooks. You finish with ten things that each work once. That is not what a portfolio looks like.

P8

## Build it

A retrieval agent that answers from a real document set.

→

P9

## Watch it

Instrument that same agent. Traces, metrics, a live dashboard.

→

P10

## Trust it

Secure and deploy that same agent. Identity, filters, governance.

THE POINT

You leave this course with one deployed, observable, secured system you can put a link to on your resume, and defend line by line. Not ten notebooks nobody will open.

THE FAILURE FIRST METHOD

# How the practicals are taught

I am not going to hand you working code and explain why it works. That produces students who can run my code and cannot write their own.

STEP 1

### The naive version

We write the obvious solution together. It looks correct. We run it.

STEP 2

### It breaks, live

It fails in front of you, in a specific way, for a specific reason. Nobody is protected from this.

STEP 3

### We fix the actual cause

Not a workaround. The real reason. And then you remember it, because you watched it hurt.

HOLD ON TO THIS

Every failure I show you is a real one from a real build. A tool that tried to do two jobs and broke on a compound instruction. A loop with no exit condition. A retrieval that answered confidently from the wrong paragraph.

SECTION 05

# Where this leads

The honest version, including the caveats.

# The roles this maps to

CLOSEST FIT

### AI Agent Developer

Builds and ships agent systems. The role this syllabus is literally benchmarked against.

ADJACENT

### AI Engineer

Broader remit. Agents are one part of a larger applied AI surface.

ADJACENT

### Automation Engineer

Enterprise workflow automation, increasingly with an agent layer on top.

LATER

### Solutions Architect

Designs the system rather than writing it. Needs the governance half of this course badly.

These are not aspirational titles I invented. They are the roles the current Microsoft certification tracks are explicitly written for.

# What an interviewer will actually test

- Can you explain why you chose an agent instead of a plain script for this problem?
- What stops your loop? Show me the termination condition.
- How do you know the answer it gave was grounded and not invented?
- What happens when the tool call fails halfway through?
- What permissions does this agent run with, and why those and not more?
- Show me a trace of one real request going through your system.

NOTICE WHAT IS ABSENT

Not one of those questions is about prompt wording. Every one of them is about engineering judgement. That is the thing this course is actually trying to build in you.

# The durable skills and the perishable ones

WILL OUTLAST THE TOOLS

## Durable

The agent loop. Tool design. When not to use an agent. How to test something non deterministic. Least privilege. Grounding and citation. Failure modes.

WILL CHANGE UNDER YOU

## Perishable

Exact SDK names. Import paths. Portal layouts. Which framework is fashionable. Product names, which Microsoft changes roughly once a year.

A REAL EXAMPLE, FROM THIS YEAR

Bot Framework SDK was the standard way to connect an agent to a chat channel. Its support ended in December 2025 and its repository is now archived. The skill of connecting an agent to a channel did not expire. The SDK did. Learn the first, expect to relearn the second.

SECTION 06

# Setup, and the money question

Because none of this matters if you cannot run it.

# Let us deal with the obvious problem first

This is a cloud course. Cloud costs money. Most of you do not have a credit card, and I am not going to pretend that is not a real obstacle.

> **Nobody in this room will be blocked from any practical for lack of money. Every single one runs on a free path, and the code is identical either way.**

That is a design decision, not a concession. It required writing every notebook a particular way, and understanding why is itself worth a few minutes.

HOW WE MAKE THE CODE PROVIDER PROOF

# The layman version

**Think of the model as electricity and your code as an appliance.**

Your mixer does not care whether the electricity came from a coal plant, a solar panel, or your neighbour's inverter. It cares about one thing: does the plug fit the socket.

Almost every model provider now offers the same socket shape. So we write every practical against that one socket, and change nothing but which plug is in the wall.

THE TECHNICAL CORRELATE

That common socket is an OpenAI compatible API. Every notebook in this course has one PROVIDER constant at the top. Change that one line and the entire notebook runs somewhere else. No other cell changes.

# What that looks like in code

```
# the only line you change all semester
PROVIDER = "github"      # foundry | github | local

client = make_client(PROVIDER)

# everything below is identical on every lane
reply = client.complete(
    model=MODEL,
    messages=[...],
    tools=[get_delivery_records],
)
```

change this,
nothing else

## Why this matters

I will demonstrate on the paid enterprise platform, because you should see the real thing.

You will run the exact same notebook on a free endpoint.

Neither of us edits the code. Same file, different socket.

# The four lanes

| | | |
|---|---|---|
| LANE A | **Microsoft Foundry** | The real enterprise platform. I demonstrate on this. You watch. My account, my cost. |
| LANE B | **GitHub Models** | Free, for every one of you, and still Microsoft infrastructure. This is your default lane. |
| LANE C | **Azure for Students** | Optional. 100 USD of credit, no credit card, verified with your university email. |
| LANE D | **Fully local** | Ollama on your own laptop. Slow and weak. Free forever. The lane that never fails. |

**HOLD ON TO THIS**

Lane B is the one to remember. You all already need a GitHub account for the course repository, so you already have the key.

YOUR DEFAULT LANE, AND THE MOST IMPORTANT SLIDE IN THIS SECTION

# Lane B, in detail

- Free API access to GPT class models, Llama, Mistral, DeepSeek and others, with a GitHub account.
- It runs on Microsoft infrastructure and speaks the OpenAI compatible protocol, so the SDK patterns match what we use on Foundry.
- No credit card. No Azure signup. No waiting on an approval from anybody.
- Endpoint: https://models.github.ai/inference

THE LIMITATION, STATED HONESTLY

Requests are capped at roughly 8K tokens in and 4K out, and the rate limits are modest. That is comfortable for learning and wrong for production. Knowing exactly where a free tier stops being adequate is itself a professional skill, so we will measure it rather than guess.

# Lane C, Azure for Students

- 100 USD in Azure credits, valid for twelve months, renewable each year while you remain a student.
- No credit card needed. You verify with your institutional email address.
- You must be eighteen or older and a full time student at an accredited degree granting institution.
- Comes with a set of always free services alongside the credit.

**TWO THINGS TO KNOW BEFORE YOU ACTIVATE**

When the 100 USD runs out the subscription is disabled rather than billed to you, which is protective, but it also means an idle resource you forgot to delete can quietly consume your whole year. The credit does not top up early. So deleting your deployed resources after every lab is a habit I will keep nagging you about.

YOUR SETUP CHECKLIST

# Before next session

- [ ] A GitHub account. You need it for the repository and for Lane B.
- [ ] Python 3.12 and VS Code installed, with the Python and Jupyter extensions.
- [ ] Git installed and configured with your name and email.
- [ ] Clone the course repository. The link is on the last slide.
- [ ] Run the setup verification script in the repo root. It checks all four lanes and prints exactly what is missing.
- [ ] Optional but recommended: activate Azure for Students now, not in October.

IF SOMETHING BREAKS

Do not spend three hours on it alone. Post the exact error text in the course channel. Half the class will hit the same error and one thread solves it for everyone.

# One naming warning, so you are not confused later

YOU WILL SEE THIS NAME

## Azure AI Foundry

In tutorials, in videos, in blog posts, and in a great deal of documentation written before 2026. Also in our own approved syllabus text.

THE CURRENT NAME

## Microsoft Foundry

Renamed at Ignite in November 2025 and formalised in the January 2026 product terms. Same platform. Same resource type. Same keys.

AND ONE MORE, FOR CONTEXT

This is the second rename in twelve months. Azure AI Studio became Azure AI Foundry, then Azure AI Foundry became Microsoft Foundry. When you search for help, search both names. This is not a mistake in the syllabus, it is the pace of the field.

SECTION 07

# How this course runs

Sessions, marks, rules, and how to get help.

# The shape of a typical session

| | | |
|---|---|---|
| OPEN | **Recall** | Five minutes on what we did last time. Not a lecture, a few questions to you. |
| CORE | **Concept** | The layman frame first, then the technical mechanism, then a worked example. |
| CORE | **Live build** | We write code together. It breaks. We fix it. Your editor should be open. |
| CLOSE | **Hold on to this** | One idea from today, stated in a single sentence, that you will need again later. |

HOLD ON TO THIS

Every session is written to stand on its own. If you miss one you will not be lost in the next, though you will have to catch up on the code.

# How you are marked

| | | |
|---|---|---|
| ATTENDANCE | 5 | Show up. Five easy marks for being in the room. |
| PRACTICALS | 45 | The ten practicals, submitted and defended. This is nearly half your grade, and it is the half you build. |
| ETP | 50 | End Term Practical. One assessment, half the marks, where you show the whole thing works and explain why. |

**THE ONE RULE THAT MATTERS**

You may use AI to write your code. You may not use it to replace your understanding. Notice that 95 of your 100 marks come from practicals and the end term practical, both of which you have to defend in person. That is where the difference shows, and it shows fast.

# Ground rules

- Ask the question. If you are confused, four other people are too, and one of them will thank you.
- Tell me when my code breaks on your machine. Environment differences are real and I want to know.
- Never commit an API key. We will cover this properly, but start the habit now.
- Delete your cloud resources after every lab. An idle deployment costs money while you sleep.
- Bring the laptop charged. Sessions with code are most of them.
- If you fall behind, say so early. Falling behind in Unit 3 is recoverable in week nine and painful in week thirteen.

HOLD ON TO THIS

And one more. This field moves fast enough that I will occasionally be wrong or out of date. When you find that, tell me. Being corrected by a student is a good day, not a bad one.

SECTION 08

# The official bits

The formal record. Credits, outcomes, textbooks, and the assessment schedule.

THE FORMAL DETAILS

# Course at a glance

| | |
|---|---|
| CODE | CSE476 |
| STRUCTURE | Lecture 3, Tutorial 0, Practical 1 |
| CREDITS | 4.0 |
| BENCHMARK | Microsoft AI Agents, From Foundations to Applications |

BOOKS

Text: Building Agentic AI Systems, by Anjanava Biswas, Packt.

Reference: Agentic AI Unleashed, by Jayaram Nanduri and Anand Oka, BPB.

THE SIX COURSE OUTCOMES

# What you will be able to do

- CO1 Build intelligent AI agents using Microsoft Foundry and modern orchestration frameworks.
- CO2 Build autonomous and conversational agents that reason, remember, plan, and use tools.
- CO3 Build production ready agents with Python, Semantic Kernel, AutoGen, and Azure services.
- CO4 Run multi agent systems for enterprise workflow automation and task execution.
- CO5 Apply prompt engineering, testing, observability, and deployment for reliable operation.
- CO6 Practice responsible AI, security, governance, and scalable deployment.

HOLD ON TO THIS

These are the promises the university holds me to. Notice every one of them starts with the word build. That is not an accident, and it is why nearly all your marks come from things you make rather than things you memorise.

ACADEMIC TASK SCHEDULE

# The three graded checkpoints

| | | | |
|---|---|---|---|
| PRACTICAL 1 | Weeks 5 to 6 | Units 1 and 2. Agents and Foundry, and building intelligent workflows. | 30 |
| PRACTICAL 2 | Weeks 10 to 11 | Units 3 and 4. Development with Python and frameworks, and multi agent systems. | 30 |
| PRACTICAL 3 | Weeks 11 to 12 | A test across Units 1 to 4. Also your chance to lift a weak earlier score. | 30 |

PLAN AROUND THESE DATES

Each is offline and individual. Practical 3 is deliberately a safety net, so a bad day earlier in the term is recoverable. But that only works if the earlier practicals are done, which is why I keep saying not to fall behind in Unit 3.

# Where everything lives

CODE

## The repository

Every notebook, every practical, the setup script, and the requirements file. Cloned once, pulled before each session.

SLIDES

## Decks and handouts

Each session ships a deck and a collapsed handout you can read without the slides.

HELP

## The course channel

Post the exact error, not "it is not working". Screenshot the full traceback. Somebody will have hit it already.

RIGHT NOW, BEFORE YOU LEAVE

Take out your phone and note the repository link on the next slide. Do the clone tonight, not on the morning of the first practical.

# What you will be able to say in December

Not "I did a course on AI agents." Anybody can say that.

> **"I built a multi agent system. Here is the live link. It is instrumented, so here is a trace of a real request. It runs with least privilege, so here is what it is not allowed to touch. It caught a prompt injection during testing, and here is the fix I wrote for it."**

That sentence is the entire point of the next fifty hours. Everything in the syllabus exists to make it true for you.

# Next session

Unit 1, Lecture 1

## What an Agent Actually Is

We take the loop you saw today apart properly. Where the boundary between a chatbot and an agent actually sits, why that boundary is blurrier than most people claim, and the four things a system needs before the word agent is honest.

COME WITH

Your environment set up and the repository cloned. We start writing code in the second half of that session.